

ICT ACADEMY OF KERALA

INTERNSHIP REPORT

FULL STACK APPLICATION DEVELOPMENT (MERN STACK)

Project Title:

Campus Complaint Management System (CCMS)

Team Members:

Arjun S

Devika V

N Vishnunandan

Institution:

CARMEL COLLEGE OF ENGINEERING AND TECHNOLOGY

Internship Duration:

1 Month

Submission Date:

8-7-2026

EXECUTIVE SUMMARY

This project titled Campus Complaint Management System (CCMS) was developed as part of the ICT Academy of Kerala Summer Internship Program. The objective of the project was to design and implement a centralized web-based platform that enables students to report campus-related issues efficiently while providing administrators with a structured system to review, monitor, and resolve complaints.

The application was built with ReactJS, Node.js, Express.js, and MongoDB. It features a responsive user interface, a secure backend, and a scalable database. Key features include user registration and authentication, complaint submission with image upload, complaint status tracking, password recovery via email verification, and an administrative dashboard for managing users and complaints. RESTful APIs were used to ensure smooth communication between the frontend and backend while keeping access to application resources secure.

The successful completion of this project provided valuable practical experience in full-stack web application development. It strengthened our understanding of frontend and backend integration, database management, API development, and collaborative software development practices. The developed system offers an efficient, transparent, and user-friendly solution for managing campus complaints, contributing to improved communication and quicker resolution of student concerns.

1. Introduction

The Campus Complaint Management System (CCMS) is a web-based application designed to simplify and improve the process of managing campus-related complaints. In many educational institutions, students report issues through manual registers, emails, or verbal communication. While these methods are commonly used, they often lead to delayed responses, poor record management, and limited transparency regarding the status of complaints. As a result, both students and administrators face difficulties in ensuring that reported issues are addressed efficiently.

To overcome these limitations, the proposed system provides a centralized platform where students can submit complaints, and track the progress of their requests in real time. Administrators are provided with a dedicated interface to review complaints, assign appropriate actions, update their status, and maintain an organized record of all reported issues. By digitizing the complaint management process, the system reduces paperwork, improves accountability, and enables faster communication between students and the administration.

The application has been designed with a strong emphasis on usability, reliability, and efficient data management. By integrating all complaint-related activities into a single platform, it minimizes manual effort and ensures that information is stored securely and can be accessed whenever required. The project demonstrates how modern web technologies can be applied to develop practical solutions that enhance administrative efficiency and contribute to a more transparent and responsive campus environment.

2. Objectives

The main objectives of the Campus Complaint Management System (CCMS) are:

- To design and develop a secure web-based platform for efficient campus complaint management.
- To provide students with a simple and convenient interface for submitting and tracking complaints.
- To enable administrators to manage, monitor, and resolve complaints through a centralized dashboard.
- To implement secure user authentication and role-based access for students and administrators.

3. Scope and Deliverables

The Campus Complaint Management System is intended to streamline the complaint handling process within an educational institution by replacing conventional manual methods with a centralized digital platform. The application allows students to register complaints, attach supporting evidence when required, and monitor the status of their submissions. Administrators can review complaints, update their progress, and maintain organized records through a dedicated management interface.

The scope of the project is limited to complaint registration, complaint management, user authentication, and status tracking. The system is designed to be scalable so that additional functionalities, such as departmental assignment, notification services, analytics, and mobile support, can be incorporated in future versions.

Deliverables

- **Web Application:** A fully functional Campus Complaint Management System developed using the MERN stack.
- **Frontend Module:** A responsive user interface developed using ReactJS for students and administrators.
- **Backend Module:** RESTful APIs and server-side functionalities developed using Node.js and Express.js.

- **Database:** A MongoDB database for storing user information, complaint records, and application data.
- **Authentication System:** Secure user login, registration, and password recovery mechanisms.
- **GitHub Repository:** A version-controlled repository containing the complete source code and project documentation.
- **Project Documentation:** A comprehensive internship report describing the project development process, implementation, and outcomes.

4. Technology Stack

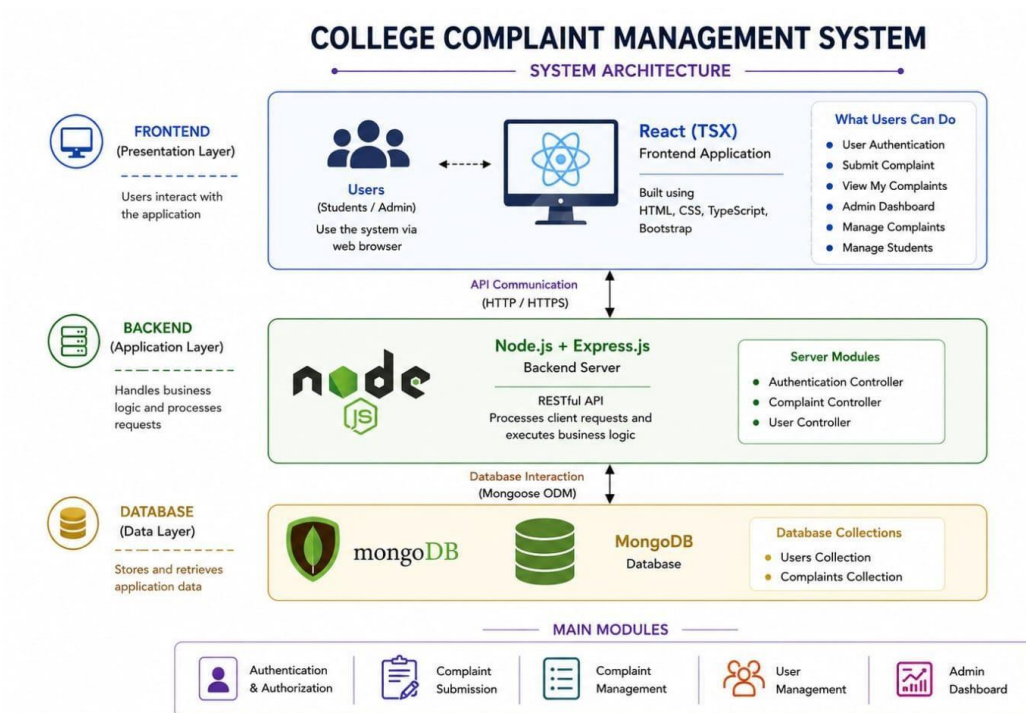
Component	Technology Used
Frontend	ReactJS
Backend	Node.js
Framework	Express.js
Database	MongoDB
Version Control	Git & GitHub

Why These Technologies?

- **ReactJS:** Used to develop a responsive and interactive user interface with reusable components.
- **Node.js:** Provides the server-side runtime environment for handling client requests efficiently.
- **Express.js:** Simplifies backend development by creating RESTful APIs and managing server-side routing.
- **MongoDB:** A flexible NoSQL database used to store and manage user and complaint data efficiently.
- **Git & GitHub:** Used for version control, code management, and collaboration throughout the project.

5. System Design

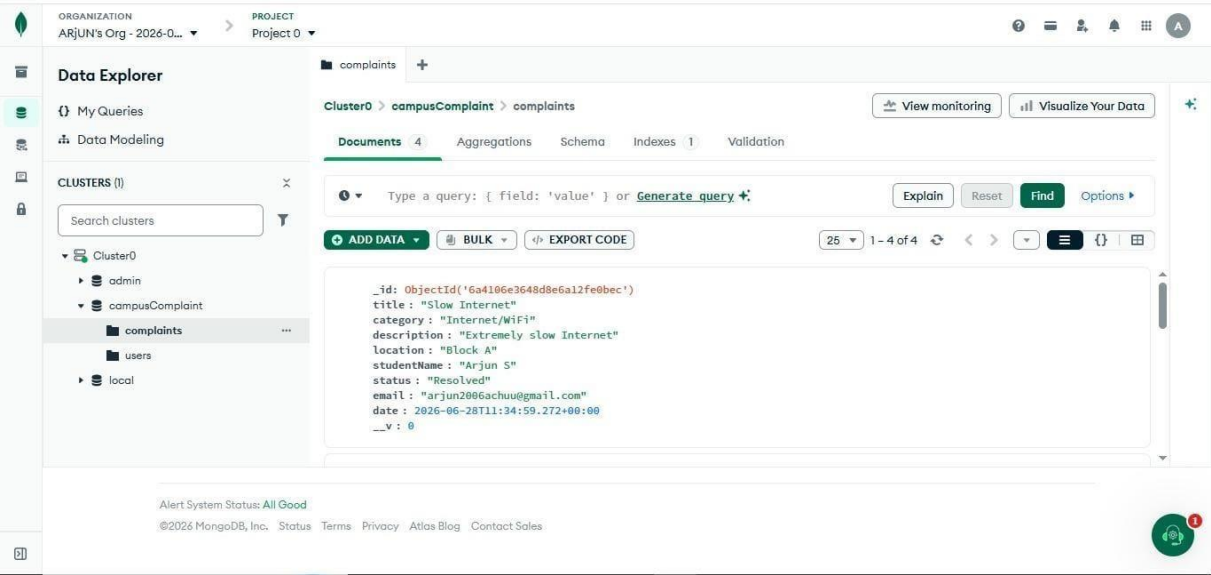
Architecture Diagram



Main Modules

Module	Purpose
Home Module	Displays the landing page and provides navigation to the application's main features.
Authentication Module	Handles user registration, login, password recovery, and secure user authentication.
Student Dashboard Module	Provides students with an overview of their activities and quick access to complaint-related features.
Complaint Management Module	Enables students to submit complaints, upload supporting images, view submitted complaints, and track their status.
Admin Dashboard Module	Allows administrators to monitor complaints, manage complaint statuses, and oversee system activities.
Student Management Module	Enables administrators to view and manage registered student information.
Database Module	Stores and manages user accounts, complaint details, and other application data using MongoDB.

Database Design



6. Features Implemented

Feature	Description
User Registration	Allows new users to create an account by entering the required details.
User Login	Authenticates users securely and provides role-based access.
Complaint Registration	Enables students to submit complaints with relevant details and supporting images.
Complaint Tracking	Allows students to view and monitor the status of their submitted complaints.
Admin Dashboard	Provides administrators with a centralized interface to manage complaints and users.
Student Management	Allows administrators to view and manage registered student records.
Password Recovery	Enables users to reset their password through email verification.
CRUD Operations	Supports creating, viewing, updating, and managing complaint records.

7. Project Activities and Implementation

Frontend Development

The frontend of Campus Complaint Management System (CCMS) was developed using ReactJS for a modern, responsive and user-friendly interface. The application is designed based on a component based architecture, which allows reusing of different UI elements in multiple pages. This approach helped to better organize the code, avoid duplication and make it easier to maintain in the future.

The project consists of several pages, each designed to perform a specific function. The Home page serves as the landing page, providing users with an overview of the application and navigation to authentication pages. The Registration page is used to register new users by providing their details. The Login page authenticates the user and allows access depending on the credentials.

After successful authentication, students will be redirected to the Student Dashboard which provides quick access to all complaint related services. Users can submit complaints on the Add Complaint page by filling in the necessary details. The My Complaints page shows all the complaints submitted by the logged-in user and their current status. This allows students to check the status of their requests without having to contact the administration directly.

A dedicated Admin Dashboard was developed for administrators to provide a centralized interface to monitor complaints and manage application data. Additional pages enable administrators to view registered students and efficiently change complaint statuses. All pages were navigated with React Router, providing a smooth single-page application experience.

Backend Development

The backend of the application was developed using Node.js and Express.js to process client requests, implement application logic, and interact with the database. RESTful APIs were designed to link the frontend with the backend to exchange data effectively.

APIs were created for user registration, login, password recovery, complaint submission, complaint status updates, and student management. Password recovery functionality was implemented to allow users to securely reset their passwords when needed.

The backend was organized using separate routes, controllers, and database models, making the code modular and easier to understand and maintain. Database operations such as creating, retrieving, and updating records were performed through these APIs, ensuring efficient data

management. The backend was designed to support the application's core functionalities while maintaining a clear and organized project structure.

Database Development

The Campus Complaint Management System uses MongoDB as its primary database to store and manage application data. MongoDB was chosen because of its flexible document-oriented structure, which makes it suitable for handling dynamic data and simplifies database management.

The database stores essential information such as user details and complaint records in separate collections. Complaint records include details such as the complaint title, description, category, status, date of submission, and the user associated with the complaint. This organized structure enables efficient storage and retrieval of information while maintaining consistency across the application.

Mongoose was used to define database schemas and facilitate communication between the backend and MongoDB. It simplified database operations such as creating new records, retrieving existing data, updating complaint information, and managing user details. The database design ensures that information is organized, easily accessible, and capable of supporting future enhancements as the application grows.

8. Challenges Faced and Solutions

Challenge 1

Implementing secure user authentication while ensuring separate access for students and administrators.

Solution:

A role-based login system was implemented, allowing users to access only the features relevant to their role after successful authentication.

Challenge 2

Establishing seamless communication between the React frontend and the Express.js backend for fetching and updating complaint data.

Solution:

RESTful APIs were developed to handle data exchange between the frontend and backend, ensuring that complaint information was displayed and updated correctly.

Challenge 3

Designing a database structure capable of efficiently storing user information and complaint records.

Solution:

Separate MongoDB collections were created for users and complaints using Mongoose schemas, enabling organized data storage and efficient retrieval.

Challenge 4

Implementing a complaint status tracking mechanism that allows students to monitor the progress of their submitted complaints.

Solution:

The complaint status was maintained in the database and updated by administrators. The frontend retrieved the latest status through API calls, allowing students to view real-time updates.

Challenge 5

Developing a secure password recovery feature for users who forgot their login credentials.

Solution:

A password recovery module was implemented to verify user identity and allow users to reset their passwords securely before regaining access to the application.

9. Results and Findings

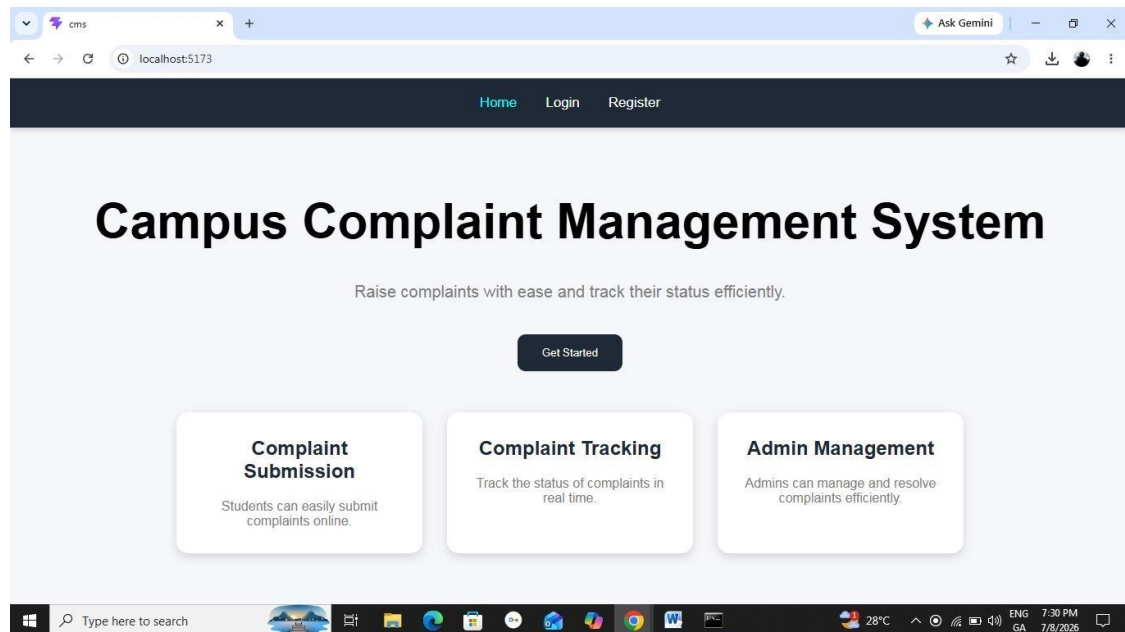


Fig 1: Home Page

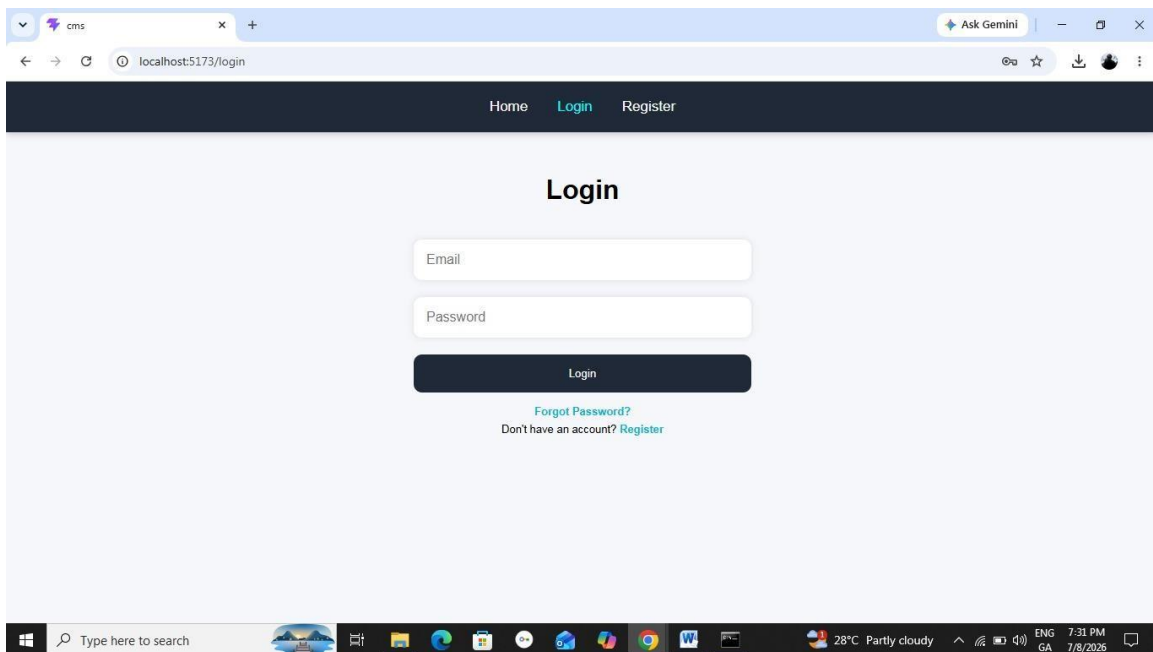


Fig 2: Login Page

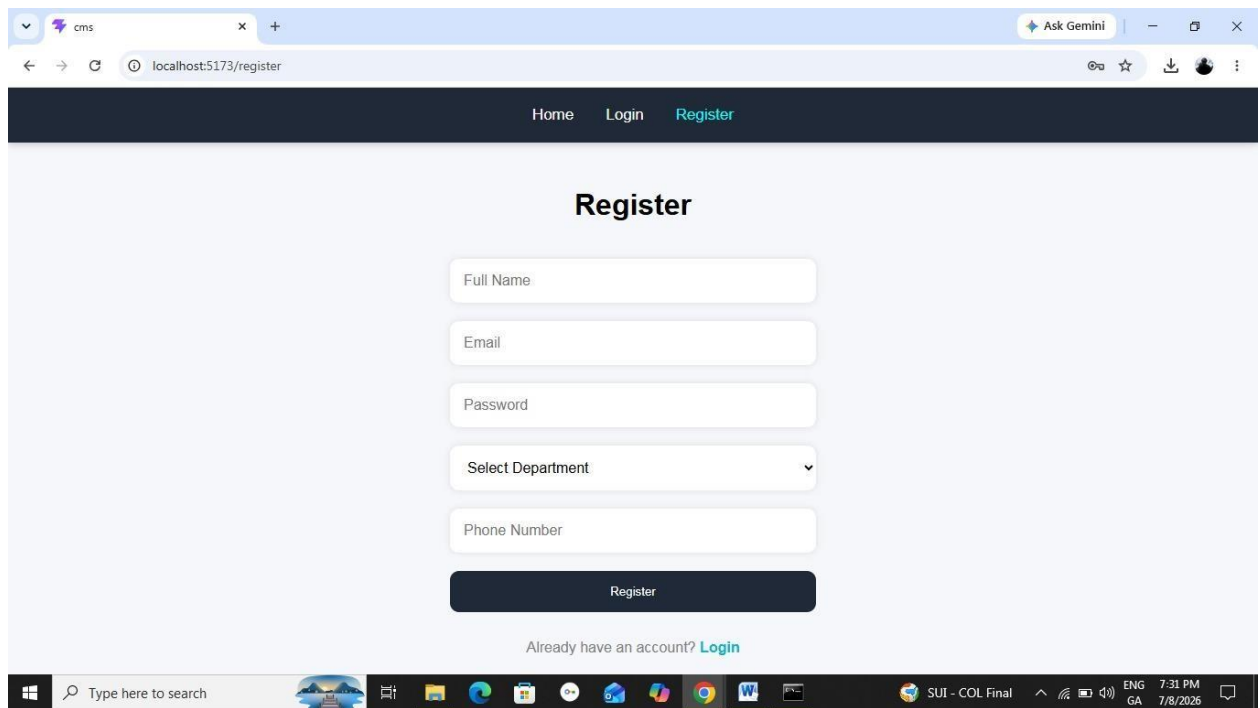


Fig 3: Register Page

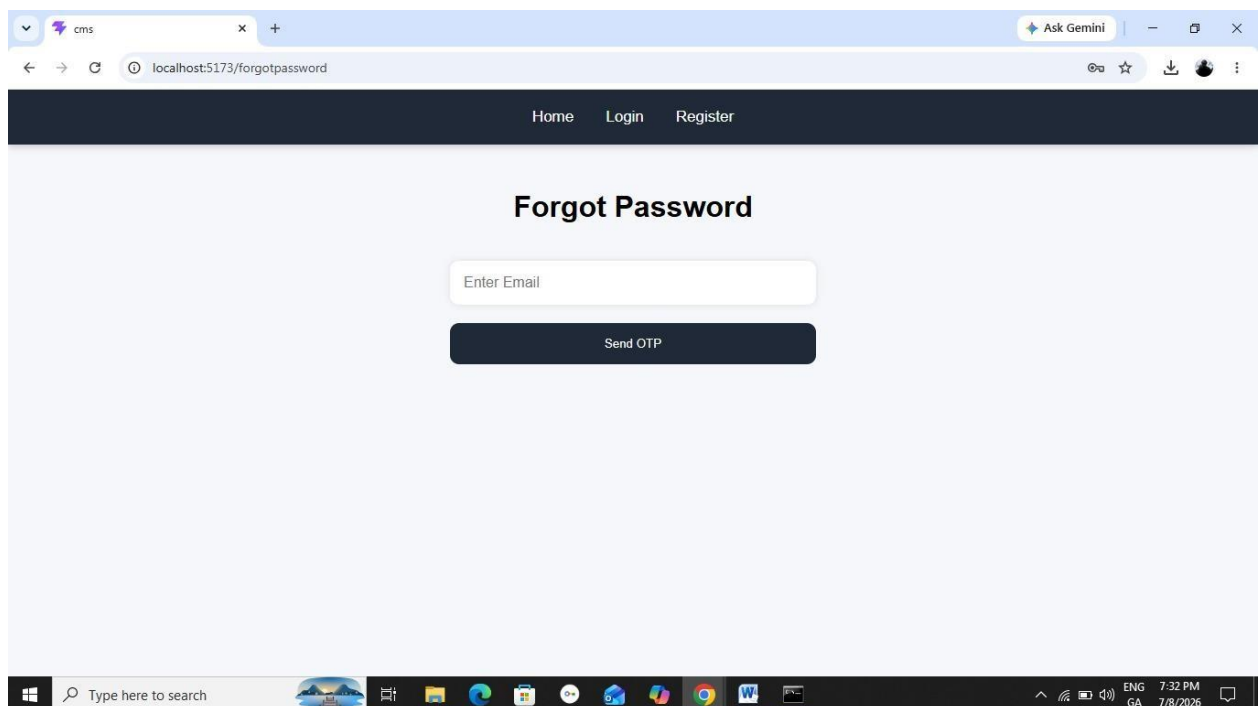


Fig 4: Forgot Password (Before OPT)

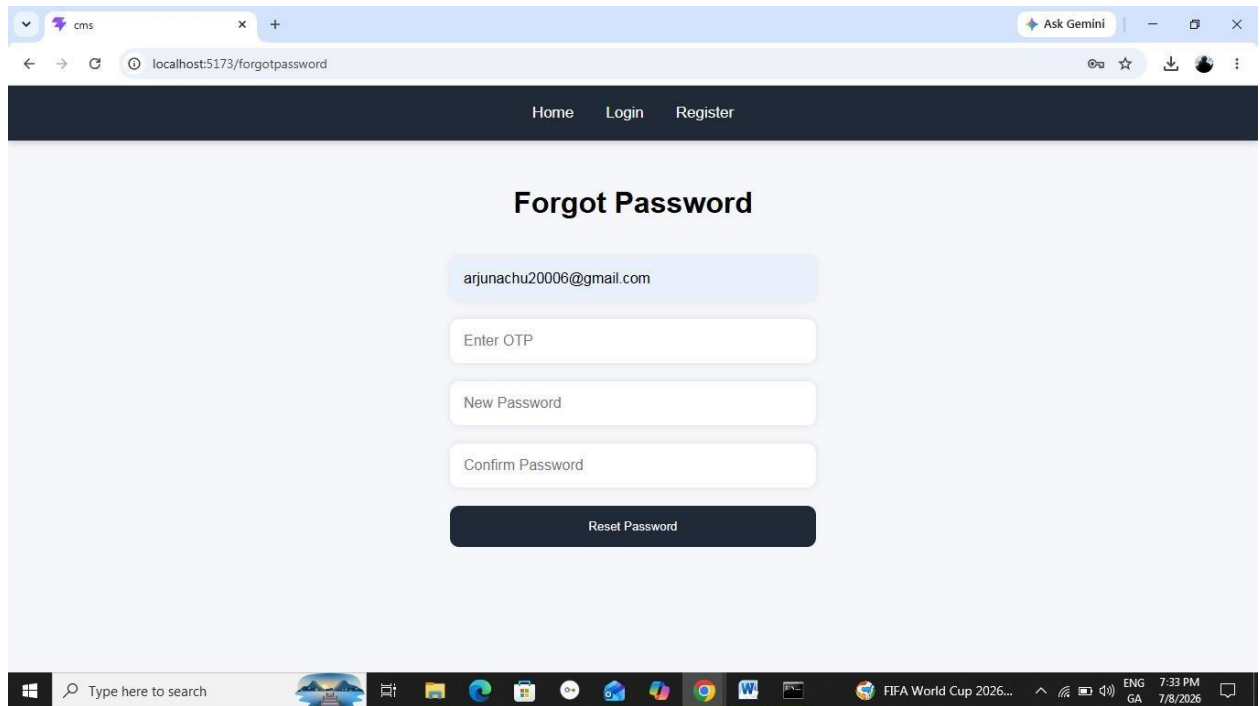


Fig 5: Forgot Password (After OPT)

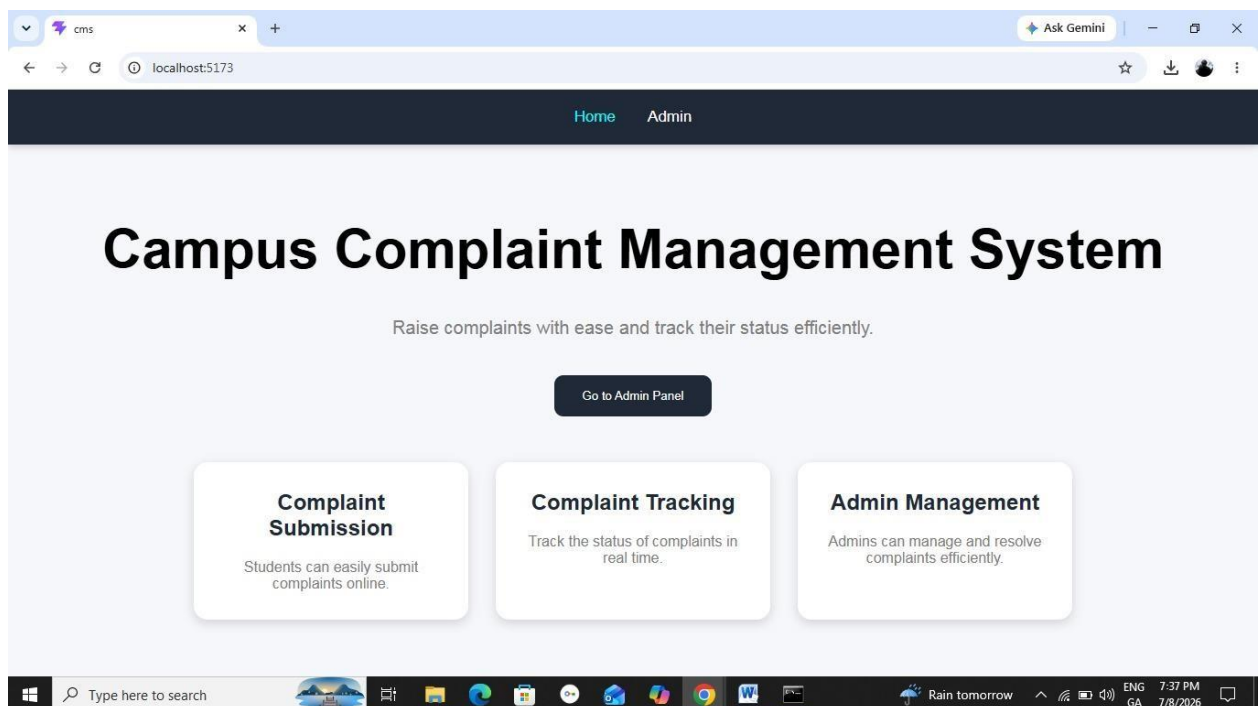


Fig 6:Admin Home Page

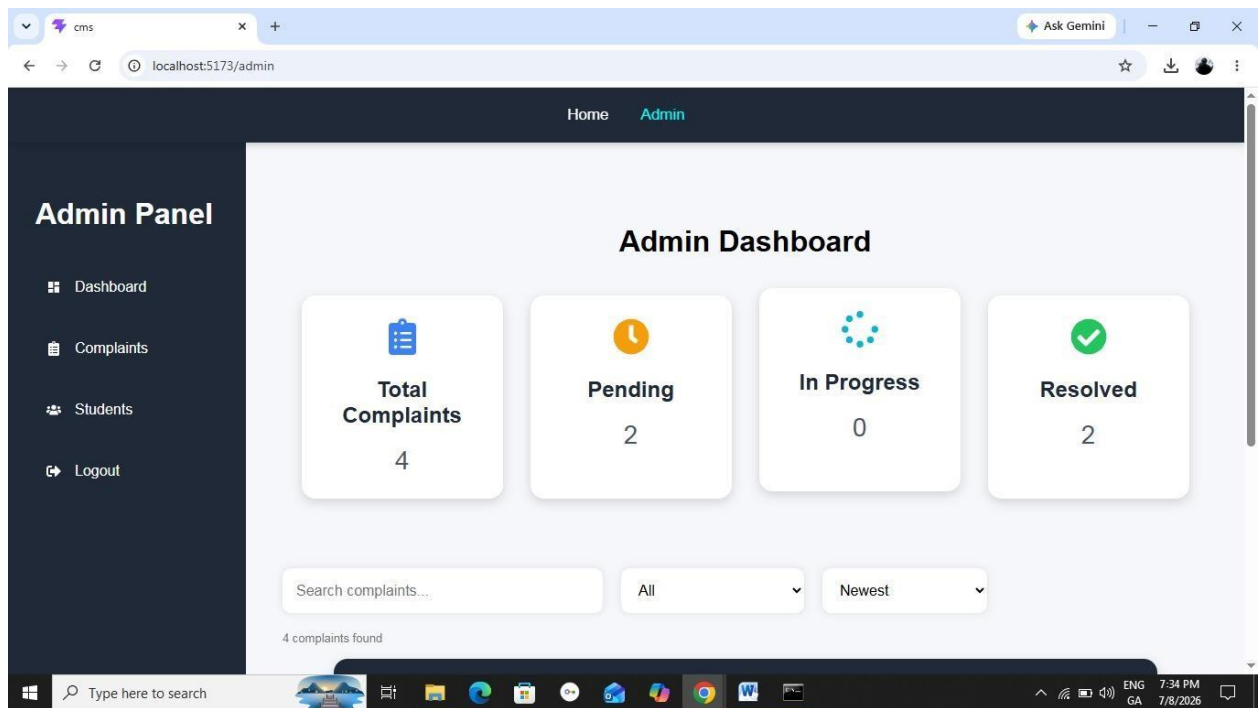


Fig 7: Admin Dashboard

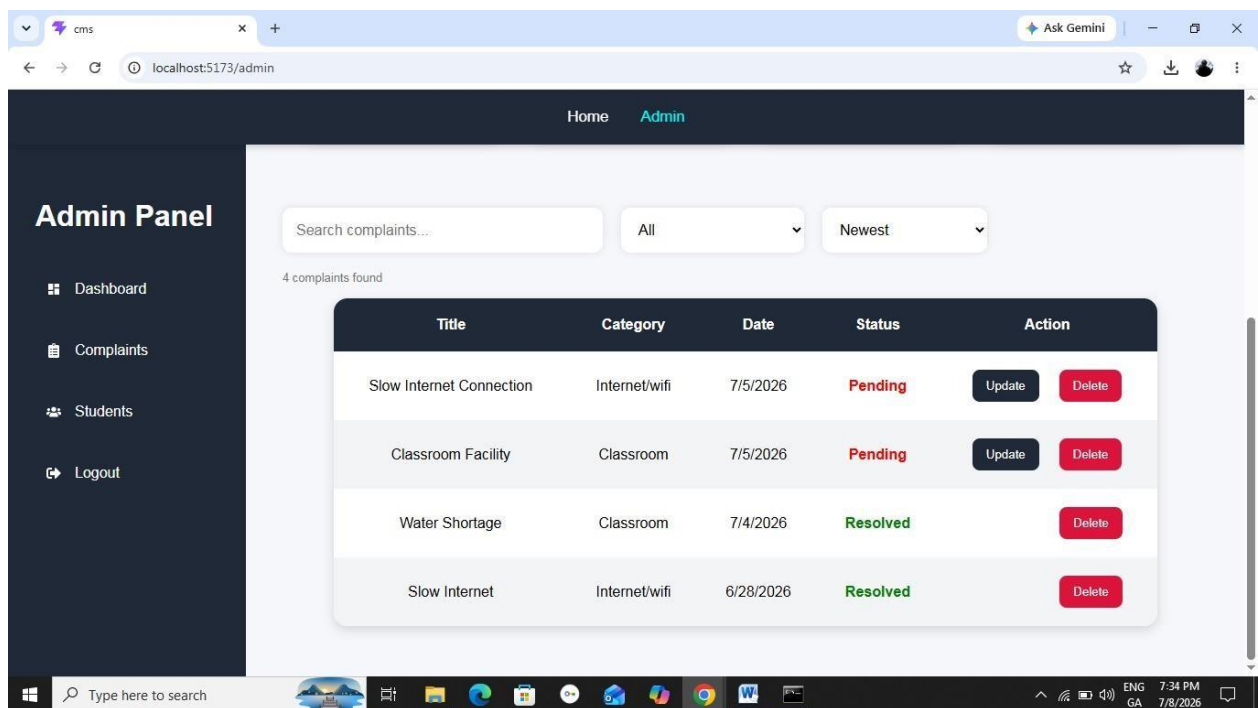


Fig 8: Complaint Board

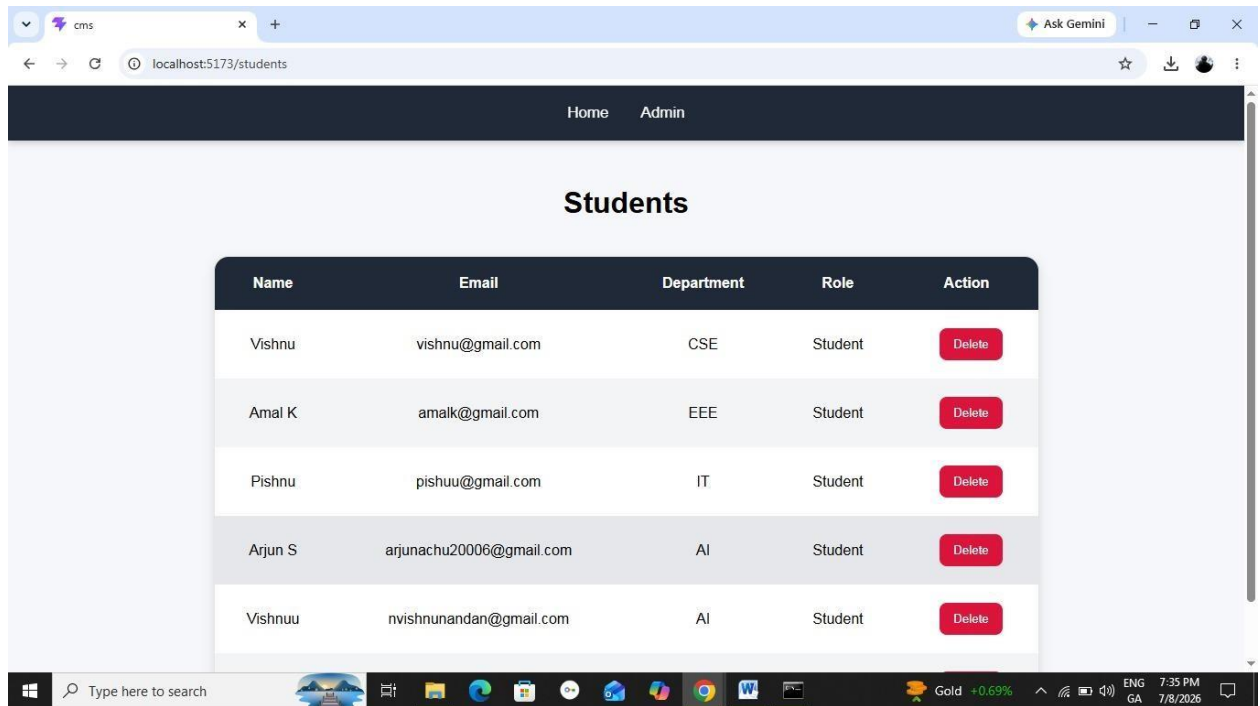


Fig 9: Student Details

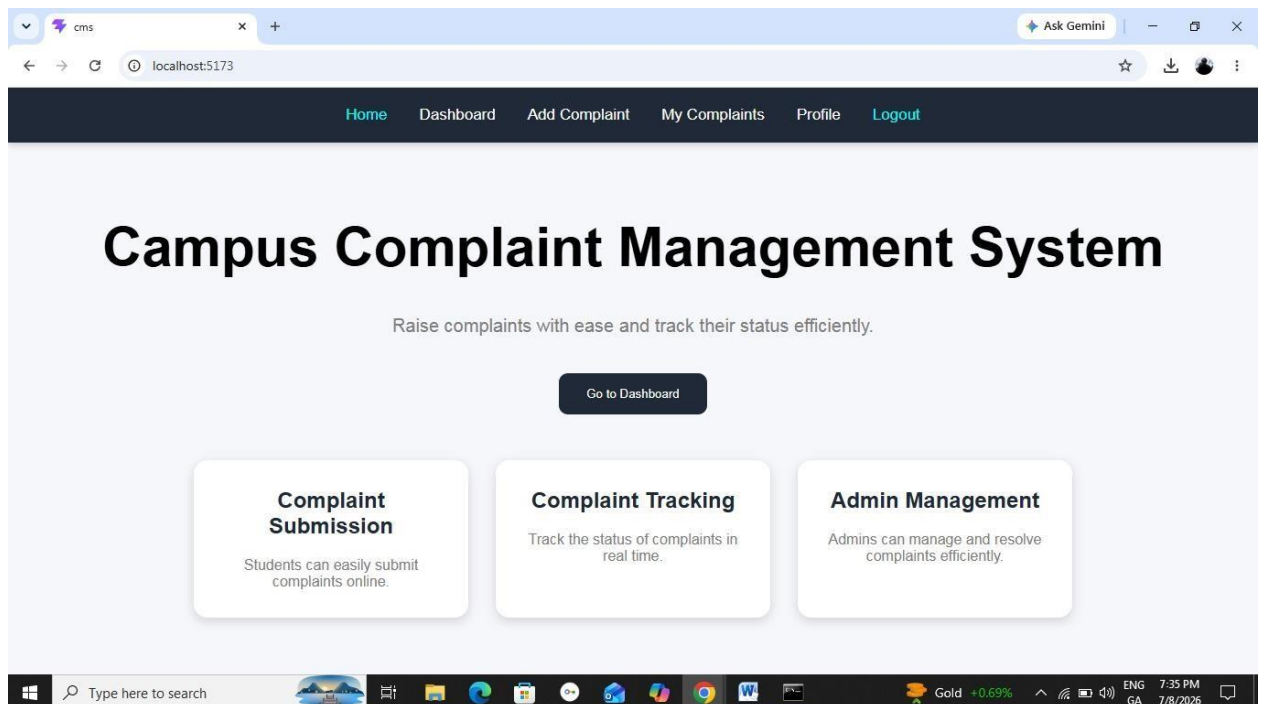


Fig 10: Student Home Page

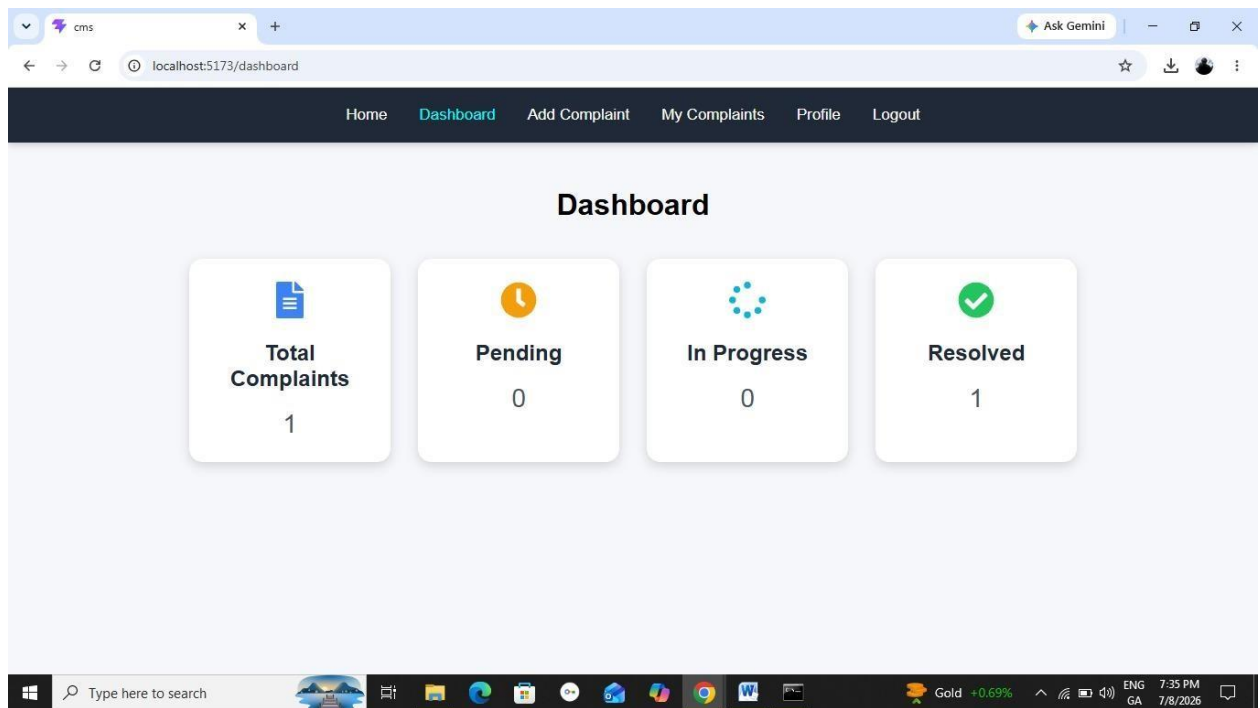


Fig 11: Student Dashboard

The screenshot shows a web browser window with the URL `localhost:5173/addcomplaint`. The navigation bar is identical to the previous figure. The main content area is titled "Add Complaint" and contains a form with the following fields:

- Title:
- Classroom:
- Describe your complaint:
- Location:

Below the form fields is a dark blue "Submit" button. The Windows taskbar and system tray are also visible at the bottom.

Fig 12: Add Complaints

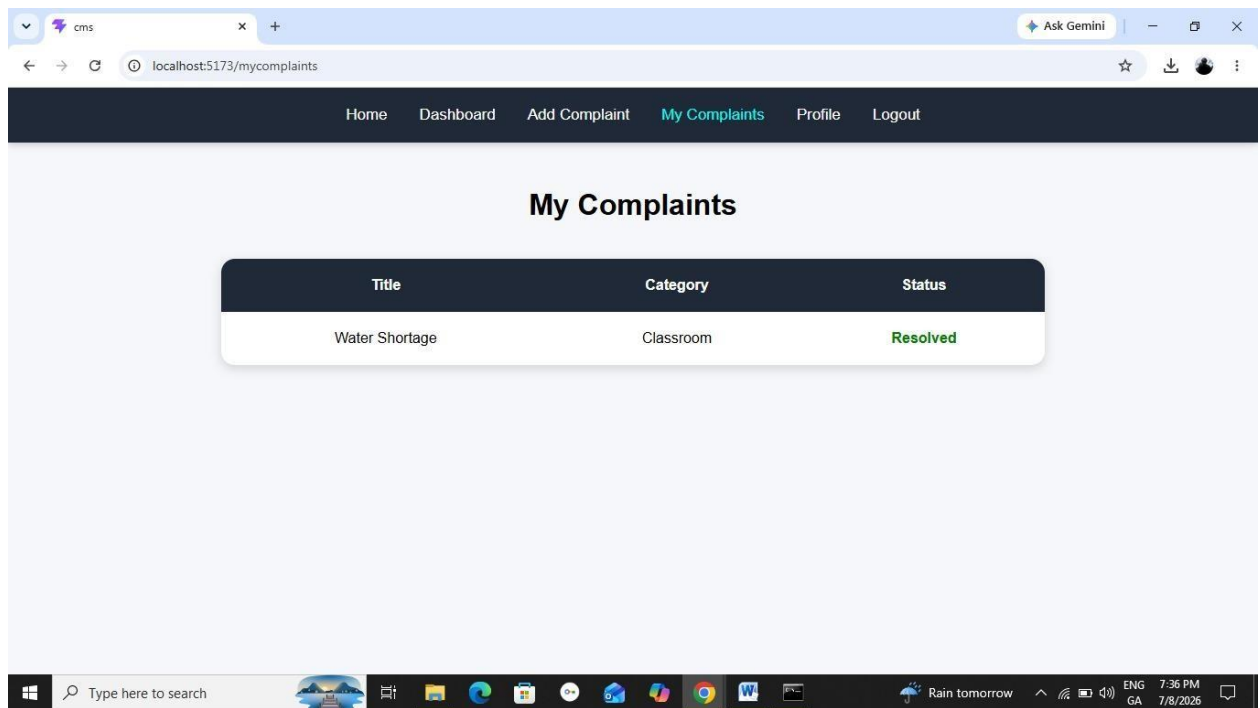


Fig 13: My Complaints

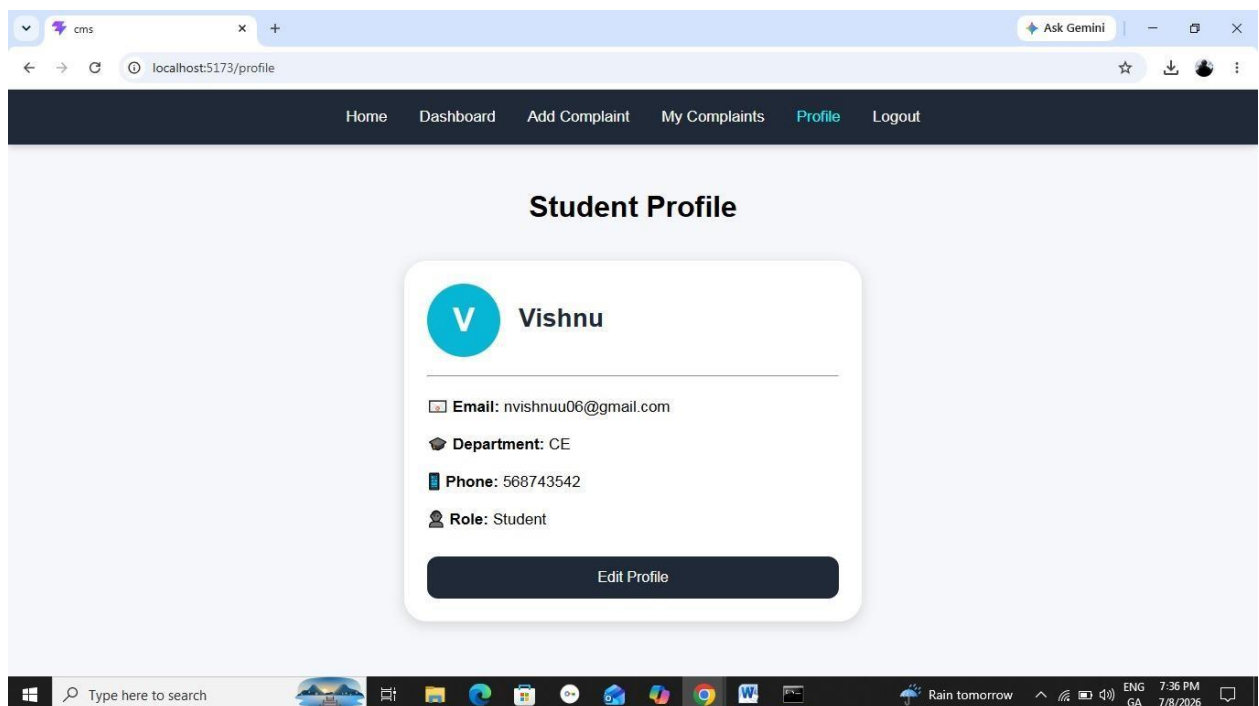


Fig 14: Student Profile

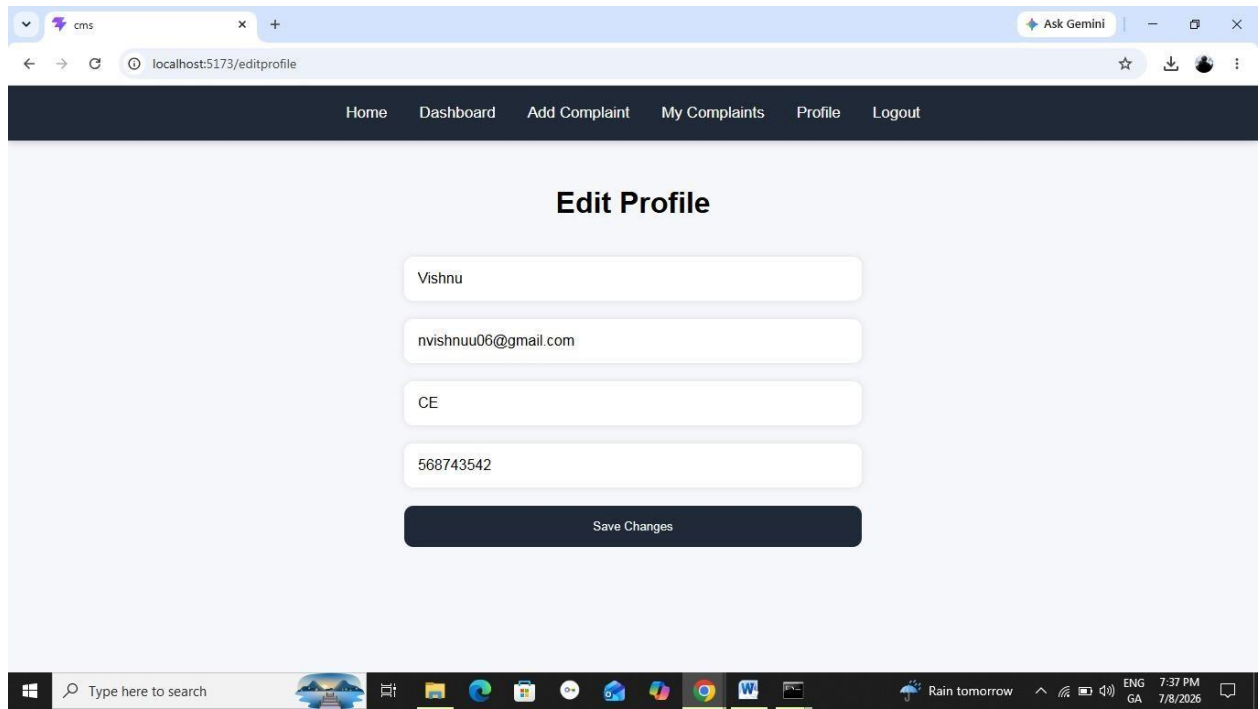


Fig 15: Edit profile

Outcome

The Campus Complaint Management System successfully replaces conventional complainthandling methods with a centralized digital platform, making the process more organized, transparent, and accessible for both students and administrators. The project strengthened our understanding of full-stack web development, improved our problem-solving and teamwork skills, and demonstrated how technology can be used to develop meaningful solutions that address real-world challenges in educational institutions.

10. GitHub Repository Details

Repository Name:

CCMS

Repository Link:

<https://github.com/toxmeic/CCMS>

Repository Description:

The GitHub repository contains the complete source code of the Campus Complaint Management System, including both the frontend and backend components. It was used for version control throughout the development process, enabling systematic code management and collaboration among team members. The repository also serves as a centralized platform for maintaining project files and tracking development progress.

Repository Structure

- Frontend Folder
- Backend Folder
- README.md
- Documentation Folder

11. Learning Outcomes

The internship provided an opportunity to apply theoretical concepts to the development of a real-world web application. Working on the Campus Complaint Management System enhanced both technical knowledge and practical software development skills. Throughout the project, valuable experience was gained in designing, developing, testing, and integrating different components of a full-stack application. The internship also strengthened teamwork, problem-solving, and project management abilities, contributing significantly to overall professional development.

The key learning outcomes are as follows:

- Gained practical experience in developing full-stack web applications using the MERN Stack.
- Improved proficiency in building responsive and interactive user interfaces using ReactJS.
- Learned to develop server-side applications using Node.js and Express.js.
- Acquired hands-on experience in designing and managing databases using MongoDB. Developed a better understanding of RESTful API development and frontend-backend integration.
- Learned to implement user authentication and password recovery functionalities.
- Improved knowledge of database operations such as creating, retrieving, updating, and managing records.
- Gained practical experience in using Git and GitHub for version control and collaborative development.
- Improved problem-solving and analytical thinking through real-world project implementation.
- Improved communication skills through regular discussions, planning, and task coordination.
- Gained confidence in applying software development concepts to build practical and user-oriented applications.

Overall, this internship was an enriching experience that not only enhanced my technical skills but also the importance of teamwork, continuous learning, and building software that solves real-world problems.

12. Conclusion

The internship offered valuable practical exposure to full-stack web application development using the MERN Stack, providing an opportunity to apply academic knowledge to a real-world software project. The development of the Campus Complaint Management System involved the complete software development process, including planning, designing, implementation, integration, testing, and documentation.

The completed application provides a centralized platform for complaint registration, tracking, and management, making the overall process more organized, transparent, and efficient. The project successfully met its intended objectives while demonstrating the effective integration of ReactJS, Node.js, Express.js, and MongoDB to build a functional web application.

Apart from gaining technical knowledge, the internship helped to understand the importance of following the correct development process, collaborating with the team, solving problems, and continuously developing in software engineering.

13. References

- React Documentation. *React: The Library for Web and Native User Interfaces*.
<https://react.dev/>
- Node.js Documentation. *Node.js Official Documentation*. <https://nodejs.org/docs/latest/api/>
- Express.js Documentation. *Express – Fast, Unopinionated, Minimalist Web Framework*.
<https://expressjs.com/>
- MongoDB Documentation. *MongoDB Official Documentation*.
<https://www.mongodb.com/docs/>
- GitHub Documentation. *GitHub Docs*. <https://docs.github.com/>
- Visual Studio Code Documentation. *Visual Studio Code Official Documentation*.
<https://code.visualstudio.com/docs>
- npm Documentation. *Node Package Manager Documentation*. <https://docs.npmjs.com/>
- HTML Living Standard. *HTML Documentation*. <https://html.spec.whatwg.org/>
- CSS Documentation. *CSS Reference – MDN Web Docs*.
<https://developer.mozilla.org/en-US/docs/Web/CSS>
- REST API Tutorial. *RESTful Web Services Guide*. <https://restfulapi.net/>
- JSON Documentation. *Introducing JSON*. <https://www.json.org/>
- Axios Documentation <https://axios-http.com/docs/intro>
- ICT Academy of Kerala. *Internship Guidelines and Learning Resources*.

DECLARATION

We hereby declare that the project work presented in this report is the outcome of our internship work carried out under the guidance of ICT Academy of Kerala. The work submitted is original and has not been copied from any source except where proper references have been provided.

Team Members:

1. Arjun S
2. Devika V
3. N Vishnunandan

Date: 8-7-2026